E-Commerce Product Entity-Resolution & Matching Platform

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Bharagava (2520090169)
Hanish (2520090107)

Team Number: 26 Section: 10

Under the guidance of

Ms K. Chandusha

Asst. Prof., CS&IT

Hyderabad-500090, Telangana, India.

ABSTRACT

This project presents the design and implementation of an E-Commerce Product Entity-Resolution and Matching Platform - a full-stack web application that identifies, deduplicates, and matches product listings referring to the same real-world entity across multiple e-commerce datasets. The system is built with a React.js frontend, a Spring Boot Java backend, and a PostgreSQL database, providing an interactive dashboard for uploading product catalogues, visualizing match results, and exploring resolved entity clusters.

The core of the platform is a custom-implemented algorithm engine embedded in the Spring Boot backend. Product title matching is performed using Knuth-Morris-Pratt (KMP) and Rabin-Karp rolling hash for fast pattern search, and Wagner-Fischer Dynamic Programming for Levenshtein Edit Distance based fuzzy title comparison. An Inverted Index with Suffix Array and LCP array performs candidate blocking, reducing the $O(n^2)$ comparison space to a manageable candidate set. Jaccard Similarity over tokenized descriptions scores token overlap between candidate pairs. A weighted similarity score is computed per pair and fed into a graph-based clustering stage where products are nodes, similarity edges connect likely matches, and Union-Find Disjoint Set Union merges them into resolved entity groups ranked by a Min-Heap priority queue. Miller-Rabin primality testing is used for safe prime selection in the double-hashing collision avoidance module.

The frontend provides real-time pipeline progress, a cluster explorer, and a side-by-side product comparison view. The backend exposes REST APIs for dataset ingestion, pipeline execution, and result retrieval. Product data is ingested from CSV uploads, preprocessed via a custom Trie-based tokenizer and HashMap stop-word filter, and results are persisted in PostgreSQL for querying and export. This project demonstrates practical integration of advanced algorithm design - string algorithms, dynamic programming, graph clustering, and randomised hashing within a production-grade full-stack architecture, solving a real industry-scale data quality problem faced by platforms such as Amazon, Flipkart, and others.

Signature of the Guide